Project Proposal: CulinaryMind AI (AI Smart Recipe & Meal Planning Agent)

1. Project Overview & Purpose

Modern household food management suffers from fragmentation, waste, and cognitive fatigue. Individuals and families frequently encounter four main problems:

- **Food Waste & Inefficient Stocking:** Ingredients expire unused due to poor tracking and lack of recipe ideas centered on available pantry stock.
- **Complex Multi-Constraint Planning:** Balancing overlapping requirements, like budget limits, macro goals, cooking skill, prep time, and allergen restrictions, is tedious and hard to execute manually.
- **Monotonous Meal Choices:** Traditional recipe search engines rely on keyword searches, failing to dynamically substitute missing ingredients or adapt to real-time kitchen availability.
- **Active Prep Disconnect:** Static recipe pages do not offer dynamic assistance during live cooking, such as step timing or instant sub-ingredient calculation.

CulinaryMind AI is an assistant designed to carry out household meal planning, grocery budgeting, and real-time cooking execution. Unlike other recipe apps that rely on static database queries or basic single-prompt LLM wrappers, CulinaryMind acts as a secretarial agent. It evaluates user dietary constraints, dynamic pantry inventories, store sale flyers, and schedule availability to formulate, adapt, and execute multi-day meal plans and step-by-step interactive cooking sessions.

2. Target Users

1. **Busy Professionals & Students:** Individuals looking to save time on weekly meal planning and grocery shopping while maintaining balanced nutrition.

2. **Budget-Conscious Household Managers:** Users seeking to minimize food waste by building plans around ingredients they already own or that are readily available in desired serving sizes.

3. **Individuals with Specific Dietary/Medical Needs:** Users managing precise dietary restrictions (e.g., keto, vegan, low-FODMAP, allergen control, PCOS) who need automated verification and recipe modifications.

3. Core System Requirements & Architectural Design

3.1 Dual Interfaces (GUI & CLI)

Both interfaces tap into the core agent execution framework via a shared Service Controller layer:

- **GUI:** A desktop interface featuring interactive meal calendar grids, visual pantry inventory dashboards, drag-and-drop recipe customization, and real-time step-by-step cooking progress displays.

- **CLI:** A terminal interface, built with Python, that enables users to perform swift commands.

3.2 AI Integration & Agent Behaviors

The system uses an LLM (OpenAI GPT or Claude 3.5 Sonnet) combined with structural tool-calling capabilities. The agent demonstrates six distinct autonomous behaviors:

1. **Reasoning & Constraint Satisfaction:** Analyzing overlapping constraints (time limits, prep complexity, nutritional macros) to evaluate meal viability.

2. **Multi-Step Planning:** Deconstructing high-level goals ("Plan meals for 5 days under $80 using items expiring soon") into structured multi-step execution graphs.

3. **Tool Use/Function Calling:** Invoking internal tools such as the Nutritional API/Calculator, Local Pantry Database, Unit Converter, and PDF/Text Grocery Exporter.

4. **Retrieval-Augmented Generation (RAG):** Querying a vector database containing verified culinary technique documents and recipe books to prevent random cooking times or invalid chemical combinations.

5. **Short-Term & Long-Term Memory:** Retaining contextual preferences across turns and learning user taste feedback, dislike patterns, and cooking skill levels over time.

6. **Self-Reflection & Plan Adjustment:** Evaluating generated plans against missing ingredients or user rejections and autonomously reformulating alternatives.

4. 10 Non-Trivial Project Features

F01 — Dynamic Pantry & Expiry Tracking

- **Description:** Manages current pantry stock, quantities, units, and expiration dates. Automatically tags items near expiry for high-priority inclusion in upcoming meal plans.
- **User Interaction (GUI):** A dedicated Pantry Dashboard table listing items with colored status indicators (Red = Expiring within 48h, Yellow = Expiring within 5 days). Includes buttons for "Add Item", "Edit Quantity", and "Delete".
- **Input:** Item name, quantity, unit, category, purchase date, and expiration date.
- **Output:** Updated inventory table and an active list of priority "Expiring Soon" ingredients.

- **AI Involvement:** Deterministic with hybrid text parsing when adding items from raw text strings.
- **Expected Workflow:**

1. User clicks "Add Pantry Item" or inputs a plain text string.
2. System validates input fields and calculates days remaining until expiry.
3. Database updates record and emits notification event to GUI pantry view.
4. Agent's internal memory updates "Expiring Ingredients" list for future planning.

- **Error/Alternative Cases:**
  - *Invalid quantity/unit input:* UI highlights field with error tooltip.
  - *Duplicate item added:* System prompts user to merge quantities or create a separate batch entry.

F02 — Multi-Day Constrained Plan Generation

- **Description:** Generates a structured multi-day meal matrix (Breakfast, Lunch, Dinner, Snacks) satisfying combined constraints (budget, calories, prep time, pantry utilization, and dietary rules).
- **User Interaction (GUI):** A "Generate Meal Plan" wizard with sliders for daily calories/macros, budget cap, target prep time, checkbox filters for dietary restrictions, and a toggle for "Maximize Pantry Usage".
- **Input:** Days (1–7), target macros, budget limit, dietary flags, and current pantry state.
- **Output:** Weekly calendar grid displaying assigned recipes, total estimated cost, macro breakdown, and pantry utilization percentage.
- **AI Involvement:** Multi-step reasoning, tool usage, RAG retrieval.
- **Expected Workflow:**

1. User configures parameters in GUI and clicks "Generate Plan".
2. Agent queries get_pantry_items() and search_recipe_vector_db() to aggregate candidate recipes.
3. Agent constructs meal matrix and runs validation tools.
4. If validation passes, plan renders in calendar view; if it fails, Agent performs self-correction.

- **Error/Alternative Cases:**
  - *Unsatisfiable Constraints (e.g., $10 budget for 7 days keto):* Agent flags impossibility, highlights tightest constraints, and suggests relaxed parameters.

F03 — Iterative Plan Refinement & Single-Meal Swapping

- **Description:** Allows selective regeneration or swapping of specific meals without disturbing the rest of the week's plan. Dependent grocery lists update automatically.
- **User Interaction (GUI):** Right-click or button press on any recipe card in the weekly menu. Buttons could be "Swap Meal" or "Modify Parameters".
- **Input:** Target meal ID, swap reason (e.g., "Too complex", "Don't like seafood"), and optional new constraints.
- **Output:** Updated meal card in weekly calendar and adjusted grocery item list.
- **AI Involvement:** Contextual replacement reasoning.
- **Expected Workflow:**

1. User selects "Swap" on Wednesday Dinner and enters feedback ("Need a 15-min option").
2. Agent retains context of already consumed/planned ingredients for other days.
3. Agent selects alternative candidate meeting the new criteria and replaces the card.

- **Error/Alternative Cases:**
  - *No alternative found matching constraints:* System alerts user and offers to relax global parameters or pull an external web recipe via search tool.

F04 — Smart Grocery Consolidation & Net Inventory Deduction

- **Description:** Calculates exact missing ingredients by subtracting existing pantry stock from total recipe requirements, converting and normalizing units (converting 3 tbsp to cups or grams) and grouping by supermarket aisle.
- **User Interaction (GUI):** "Grocery List" tab showing checklist grouped by category (Produce, Dairy, Meat, Spices) with toggle switches for "In Stock" vs "To Buy".
- **Input:** Selected meal plan structure and current pantry inventory database.
- **Output:** Itemized grocery list with quantities, units, aisle labels, and estimated cost.
- **AI Involvement:** Deterministic unit conversions and LLM semantic entity resolution for non-standard item descriptions.
- **Expected Workflow:**

1. User opens "Grocery List" tab after generating/modifying a meal plan.
2. System aggregates ingredient requirements across all selected plan recipes.
3. Unit normalization module converts quantities into common base units.

- **Error/Alternative Cases:**
  - *Ambiguous Unit Conversion (e.g., "1 bunch of parsley" vs "100g parsley"):* LLM fallback estimates standard weight equivalent and adds an editable badge for user confirmation.

F05 — Interactive Real-Time Cooking Mode

- **Description:** Provides step-by-step cooking instruction card views, multi-timer tracking, and an interactive assistant for live, mid-cook substitutions and technique guidance.
- **User Interaction (GUI):** Clicking "Start Cooking" on a recipe opens a focused step view with large text, active timer buttons, and a text/voice query drawer ("Ask Kitchen Assistant")
- **Input:** Active recipe ID, current step index, user mid-cook query ("I ran out of butter, what can I use?").
- **Output:** Step-by-step progress cards, active count-down timers, and instant substitution suggestions scaled to recipe portions.
- **AI Involvement:** Real-time sub-task breakdown & dynamic ingredient substitution reasoning.
- **Expected Workflow:**

1. User enters cooking mode for "Chicken Alfredo".
2. Step 1 displays prep instructions; clicking "Start 5m Timer" launches an asynchronous UI countdown.
3. User asks: "My cream sauce is separating."

4. Agent queries RAG culinary techniques database and returns instant fix instructions.

- **Error/Alternative Cases:**
  - *Timer expires:* Audio alarm triggers, popup alerts user, and system auto-advances or prompts for next step confirmation.

F06 — Recipe Adaptation & Allergen Sanitizer

- **Description:** Converts any standard input recipe into an allergen-safe, diet-compliant variant while preserving flavor balance and recalculating prep times and nutrition.
- **User Interaction (GUI):** Import/Paste recipe URL or select target modification checkboxes ("Make Gluten-Free", "Low Sodium", "Vegan").
- **Input:** Raw recipe text/structured recipe and target restriction flags.
- **Output:** Adapted recipe title, modified ingredient list (with substituted quantities), updated instructions, and safety verification badge.
- **AI Involvement:** Context-aware culinary substitution and chemical/functional ingredient replacement.
- **Expected Workflow:**

1. User pastes a classic Lasagna recipe and checks "Gluten-Free" and "Dairy-Free".
2. Agent scans ingredients against allergen databases.
3. Agent replaces wheat pasta with brown rice noodles and mozzarella with cashew-based alternatives.
4. Agent adjusts baking temperature/time adjustments if necessary and presents adapted recipe.

- **Error/Alternative Cases:**
  - *Core item cannot be safely substituted without destroying recipe:* Agent explicitly warns user (e.g., "Angel Food Cake requires egg whites; vegan substitute will change texture significantly").

F07 — Macro & Micronutrient Deep-Dive Analysis

- **Description:** Computes detailed breakdown of daily/weekly macro-nutrients and micronutrients against standard RDA guidelines.
- **User Interaction (GUI):** "Nutritional Insights" tab featuring interactive pie charts and bar graphs.
- **Input:** Active meal plan and user demographic/fitness target profile (Age, Weight, Activity level, Goal).
- **Output:** Nutritional graphs, percentage score of compliance, and nutrient deficiency/excess warnings.
- **AI Involvement:** Deterministic calculation via USDA/Nutritional database and LLM narrative interpretation and advice.
- **Expected Workflow:**

1. User opens "Nutritional Insights" tab for the current week.
2. Deterministic calculator sums nutrients per day across all scheduled recipes.

3. System renders progress charts against target profile RDA.
4. LLM analyzes gaps and generates a concise summary ("Day 3 is low in Fiber, consider adding chia seeds to breakfast").

- **Error/Alternative Cases:**
  - *Missing nutritional data for custom ingredient:* System prompts user to estimate values or lets LLM approximate based on similar food items.

F08 — Pantry Import via Receipt / Unstructured Text Parser

- **Description:** Parses ingredient lists, pasted grocery receipts, or natural language prompts to automatically extract, categorize, and load items into the pantry.
- **User Interaction (GUI):** "Import Pantry" dialog box with a text box for pasting receipt text/notes and an "Extract & Add" button.
- **Input:** Unstructured text input.
- **Output:** Structured preview list of detected items, quantities, categories, and estimated expiry dates ready for 1-click confirmation.
- **AI Involvement:** Named Entity Recognition, quantity/unit extraction, category classification.
- **Expected Workflow:**

1. User pastes raw text block into receipt import dialog.
2. LLM tool parses entities into JSON format.
3. GUI renders an interactive preview table allowing user to edit parsed fields.
4. User clicks "Confirm Import", committing items to the pantry database.

- **Error/Alternative Cases:**
  - *Garbled/Unclear receipt line:* System marks item as "Uncategorized / Review Required" in preview table for manual correction.

F09 — Export & Integration Hub

- **Description:** Exports generated meal plans and grocery lists to external formats including JSON, CSV, PDF, formatted Markdown, and .ics calendar files.
- **User Interaction (GUI):** "Export Plan" dropdown button on the top action bar offering options: "Export to PDF", "Export Grocery CSV", "Sync to Calendar (.ics)".
- **Input:** Selected meal plan object, grocery list object, and chosen target export format.
- **Output:** Formatted document file saved to local filesystem or downloadable file buffer.
- **AI Involvement:** Deterministic (File formatting and schema export generators).
- **Expected Workflow:**

1. User selects "Sync to Calendar (.ics)" from the export menu.
2. System transforms meal plan schedule into standard iCalendar with recipe names and prep links in descriptions.
3. File save dialog opens, allowing user to save meal_plan_week4.ics.

- **Error/Alternative Cases:**

- *File write permission denied:* System catches OS exception and prompts user to select an alternative destination folder.

F10 — Persistent User Feedback & Taste Profiler

- **Description:** Captures post-meal ratings and natural language feedback, updating a persistent long-term memory profile that shapes future planning decisions.
- **User Interaction (GUI):** Rating dialog on completed meal cards featuring a 5-star rating control, difficulty tag, and a text box for notes.
- **Input:** Recipe ID, numerical rating (1–5), prep effort rating, and free-form feedback text.
- **Output:** Updated preference database record and dynamic updates to the agent's long-term memory system prompt context.
- **AI Involvement:** Sentiment analysis, preference extraction, and long-term memory profile updates.
- **Expected Workflow:**

1. User marks a cooked recipe as "Finished" and opens the feedback popup.
2. User enters 2 stars and writes: "Cilantro flavor was overpowering."
3. Agent extracts preference rule: Dislike: Heavy Cilantro / Action: Reduce or omit cilantro in future recipes.
4. Rule is saved to user profile database and loaded into context during future F02/F03 runs.

- **Error/Alternative Cases:**
  - *Conflicting feedback (liked dish earlier, disliked later):* Agent weighs recent feedback higher and asks clarifying preference questions on next plan generation.

5. Software Design Patterns Involved

To ensure high architectural quality and maintainability, the system incorporates five established design patterns:

1. **Model-View-Controller (MVC) / Model-View-Presenter (MVP):**
   a. *Purpose:* Decouples core agent domain logic and persistent data from both the GUI and CLI presentation layers, unified by shared presentation controllers.

2. **Strategy Pattern:**
   a. *Purpose:* Swaps meal-planning optimization algorithms dynamically depending on user configuration.

3. **Factory Method Pattern:**
   a. *Purpose:* Instantiates system tools and command parsers

4. **Observer Pattern:**

a. *Purpose:* Maintains real-time state synchronicity between the GUI dashboard, CLI context, and underlying persistent databases. For example, when a recipe is cooked, the pantry model notifies observers to decrement stock levels immediately.

5. **Chain of Responsibility Pattern:**
   a. *Purpose:* Processes user input and generated plans through a pipeline of validation filters before presenting results to the user.

1.6 Model Interaction Architecture

The system employs a decoupling pattern where the LLM operates inside an Agent Controller layer.

User Interfaces (GUI / CLI)
|
v

Service Controller Layer
|
v

Agent Execution Engine

(Reasoning Loop, Memory & Self-Reflection)
| | |
v v v
LLM API ChromaDB System Tools

(Function Calls) (RAG Store) (Pantry, DB, Export, Calcs)

Task 2: System Architecture & UML Design

2.1 Class Diagram & Design Pattern Analysis

2.1.1 Overview of Design Patterns Applied

Pattern	Design Problem Addressed	Participating Classes	Class Roles	Why Appropriate	Difficulty Without Pattern
Model-View-Controller (MVC)	Decoupling domain/agent state from presentation across	MealPlanModel, PantryModel, MainGUIView, CLIView, ServiceController	• **Model:** Domain data & business rules.	Permits desktop GUI and terminal CLI to share	High coupling between UI code and business logic; adding or modifying UI

	GUI and CLI.		• **View:** UI display rendering. • **Controller:** Maps user actions to model & agent operations.	identical agent logic without duplicate code.	views requires refactoring LLM code.
Strategy	Dynamically altering meal planning optimization rules based on user goals.	PlanStrategy (Interface), BudgetFirstStrategy, ZeroWasteStrategy, HighProteinStrategy	• **Strategy:** Defines algorithm interface. • **Concrete Strategies:** Implement specific scoring/filtering heuristics.	Users require different planning goals (e.g., saving money vs using expiring pantry items).	Monolithic if-else or switch blocks in the agent generator; adding a new goal breaks Open/Closed Principle.
Factory Method	Encapsulating creation of complex external tools for agent tool-calling.	ToolFactory (Abstract), SystemToolFactory (Concrete), BaseTool, PantryLookupTool, MacroCalcTool, RAGSearchTool	• **Creator:** Declares tool instantiation method. • **Concrete Creator:** Instantiates specific tool objects with DB dependencies. • **Product:** Abstract tool wrapper for	Standardizes how agent tools are initialized with configuration, database handles, and API keys.	Scattered initialization logic across the codebase; tight coupling between the agent engine and concrete tool implementations.

			LLM function calling.		
Observer	Synchronizing UI displays when background agent operations modify local database state.	Subject (Interface), Observer (Interface), PantryModel, MealPlanModel, PantryDashboardView, CalendarView	• **Subject:** Maintains state and notifies listeners. • **Observer:** Implements update() to refresh UI views.	Modifying pantry stock (e.g., during live cooking) automatically reflects across GUI tables and CLI states.	Manual polling or hardcoding explicit refresh calls from domain models directly into UI widgets.
Chain of Responsibility	Validating generated meal plans against strict sequential constraints before showing them to the user.	PlanValidationHandler (Abstract), AllergenCheckHandler, BudgetCheckHandler, MacroCheckHandler	• **Handler:** Declares interface for processing plan validation. • **Concrete Handlers:** Evaluate specific safety or nutritional rules.	Ensures generated plans undergo multi-stage safety (allergens) and financial (budget) checks with early exit on failure.	Massive, rigid validation routines that are difficult to reorder, extend, or unit test independently.

2.1.2 Class Diagram Description & Structure

The class diagram below captures the structural layout of CulinaryMind AI, bridging standard MVC software architectures with autonomous AI Agent components:

- **Presentation Layer:** Main GUI View and CLI View implement the Observer interface to receive real-time state updates from models. They forward user actions to ServiceController.
- **Controller & Agent Execution Layer:** ServiceController orchestrates requests and delegates complex agent tasks to AgentExecutionEngine. The engine manages short-term conversational context, persists user taste habits, and executes a reasoning loop using LLMProviderAdapter.

- **Tool Framework:** AgentExecutionEngine requests tools via SystemToolFactory, which instantiates PantryLookupTool, MacroCalcTool, and RAGSearchTool derived from BaseTool.

- **Domain Models & Validation:** MealPlanModel uses PlanStrategy implementations to shape constraints and delegates output through a PlanValidationHandler pipeline.

Use Case Descriptions

UC01: Manage Pantry Inventory

- **Use Case ID:** UC01
- **Use Case Name:** Manage Pantry Inventory
- **Actor:** Primary User
- **Goal:** Create, read, update, or delete items in the local pantry database and track expiration dates.
- **Preconditions:** System database initialized; GUI or CLI active.
- **Trigger:** User opens the Pantry tab or issues a pantry command in CLI.
- **Main Success Scenario:**
    - User enters item details (name, quantity, unit, expiration date).
    - System validates input types and writes the item to the database.
    - PantryModel notifies all registered observers.
    - Interface updates tables with color-coded expiry alerts (Red = Expiring within 48 hours).
- **Alternative/Exception Flows:**
    - *A1 (Invalid Quantity/Unit):* System displays validation errors and highlights input fields.
- **Postconditions:** Database record created/updated; pantry memory context refreshed.
- **Related Features:** F01 — Dynamic Pantry & Expiry Engine.

UC02: Generate Multi-Day Meal Plan

- **Use Case ID:** UC02
- **Use Case Name:** Generate Multi-Day Meal Plan
- **Actors:** Primary User, External AI Services (LLM)
- **Goal:** Autonomously create a multi-day meal schedule satisfying budget, dietary, macro, and pantry utilization constraints.
- **Preconditions:** Pantry inventory populated (optional); user preferences configured.
- **Trigger:** User clicks "Generate Plan" in GUI or executes culinary agent plan in CLI.
- **Main Success Scenario:**
    - User specifies plan duration (1–7 days), macro targets, budget ceiling, and strategy selection.
    - ServiceController passes parameters to AgentExecutionEngine.
    - Agent invokes PantryLookupTool and RAGSearchTool to aggregate recipes.
    - Agent formulates schedule and passes output through PlanValidationHandler pipeline.
    - Plan renders in GUI Calendar grid / CLI text summary.
- **Alternative/Exception Flows:**

    - *A1 (Validation Failure - Over Budget/Allergen Conflict):* Chain of responsibility rejects plan; Agent enters self-reflection loop to adjust recipe choices up to 3 times.
    - *A2 (Impossible Constraints):* Agent terminates loop and presents explanation with suggested parameter relaxations.
- **Postconditions:** Validated MealPlanModel saved and rendered.
- **Related Features:** F02 — Multi-Day Constrained Plan Generation.

UC03: Swap / Refine Single Meal

- **Use Case ID:** UC03
- **Use Case Name:** Swap / Refine Single Meal
- **Actors:** Primary User, External AI Services (LLM)
- **Goal:** Replace a single scheduled meal with a fresh alternative without re-generating the entire week.
- **Preconditions:** Active meal plan exists in MealPlanModel.
- **Trigger:** User selects "Swap Meal" on a specific calendar slot and inputs feedback.
- **Main Success Scenario:**
    - User selects target meal slot and enters feedback ("Need a 15-minute quick option").
    - Agent receives context of surrounding meals to avoid repetition.
    - Agent selects an alternative recipe matching the new constraint.
    - System updates target calendar slot and re-calculates net grocery list requirements.
- **Alternative/Exception Flows:**
    - *A1 (No Matching Alternative):* Agent offers option to pull an external recipe via web search tool or relax constraints.
- **Postconditions:** Updated meal slot in MealPlanModel; consolidated grocery list adjusted.
- **Related Features:** F03 — Iterative Plan Refinement & Single-Meal Swapping.

UC04: Consolidate Smart Grocery List

- **Use Case ID:** UC04
- **Use Case Name:** Consolidate Smart Grocery List
- **Actor:** Primary User
- **Goal:** Generate a net shopping list by subtracting pantry stock from active recipe requirements and grouping by supermarket aisle.
- **Preconditions:** Active meal plan generated in MealPlanModel.
- **Trigger:** User opens the "Grocery List" tab or executes culinary-agent grocery.
- **Main Success Scenario:**
    - System aggregates total required ingredients across scheduled meals.
    - Unit normalization engine converts varying units (e.g., tbsp to cups) to base metrics.
    - Deductor subtracts available pantry quantities from totals.
    - System groups remaining items by aisle (Produce, Dairy, Meat) and renders actionable checklist.

- **Alternative/Exception Flows:**
  - *A1 (Non-standard units):* System delegates entity normalization to LLM tool fallback.
- **Postconditions:** Net grocery list rendered with interactive checkboxes.
- **Related Features:** F04 — Smart Grocery Consolidation & Net Inventory Deduction.

UC05: Interactive Live Cooking Assistant

- **Use Case ID:** UC05
- **Use Case Name:** Interactive Live Cooking Assistant
- **Actor(s):** Primary User, External AI Services (LLM)
- **Goal:** Guide user step-by-step through recipe preparation with dynamic timers and instant ingredient substitutions.
- **Preconditions:** Recipe selected from meal plan.
- **Trigger:** User clicks "Start Cooking".
- **Main Success Scenario:**
  - System opens interactive step-by-step view.
  - User activates asynchronous kitchen timers directly from step instructions.
  - User submits mid-cook substitution query ("Out of buttermilk").
  - Agent calculates equivalent sub-ratio and displays response.
- **Alternative/Exception Flows:**
  - *A1 (Timer expires):* System triggers audio/visual alarm and pauses progression until acknowledged.
- **Postconditions:** Recipe marked as completed; prompt to log post-meal feedback.
- **Related Features:** F05 — Interactive Real-Time Cooking Mode.

UC06: Adapt Recipe & Sanitize Allergens

- **Use Case ID:** UC06
- **Use Case Name:** Adapt Recipe & Sanitize Allergens
- **Actors:** Primary User, External AI Services (LLM)
- **Goal:** Convert an existing recipe into a safe variant respecting specified dietary or allergen flags.
- **Preconditions:** User imports/pastes a recipe.
- **Trigger:** User selects "Adapt Recipe" and checks allergen flags (e.g., "Gluten-Free").
- **Main Success Scenario:**
  - Agent scans ingredient list against allergen vector database.
  - Agent replaces flagged ingredients with safe alternatives.
  - Agent adjusts cooking parameters (time/temp) impacted by substitution.
  - Adapted recipe renders with explicit safety badges.
- **Alternative/Exception Flows:**
  - *A1 (Structural ingredient missing):* Agent issues warning that substitution will significantly alter texture/flavor.
- **Postconditions:** Modified recipe saved to library.
- **Related Features:** F06 — Recipe Adaptation & Allergen Sanitizer.

UC07: View Macro & Micronutrient Deep-Dive

- **Use Case ID:** UC07
- **Use Case Name:** View Macro & Micronutrient Deep-Dive
- **Actor:** Primary User

- **Goal:** Calculate and visualize total nutritional profile of a plan against recommended daily allowances (RDA).
- **Preconditions:** Active meal plan exists.
- **Trigger:** User selects "Nutritional Insights" tab.
- **Main Success Scenario:**
    - System sums macros and micronutrients across scheduled meals via MacroCalcTool.
    - GUI renders macro pie charts and RDA percentage bar graphs.
    - Agent provides short narrative analysis highlighting potential nutrient deficits.
- **Alternative/Exception Flows:**
    - *A1 (Missing nutrition data for custom item):* System estimates values using LLM food knowledge base.
- **Postconditions:** Visual charts and narrative assessment rendered.
- **Related Features:** F07 — Macro & Micronutrient Deep-Dive Analysis.

UC08: Import Pantry via Receipt / Text Parser

- **Use Case ID:** UC08
- **Use Case Name:** Import Pantry via Receipt / Text Parser
- **Actor(s):** Primary User, External AI Services (LLM)
- **Goal:** Parse unstructured text or receipts to automatically extract and populate pantry items.
- **Preconditions:** Raw receipt text or notes available.
- **Trigger:** User pastes text into "Import Receipt" dialog and clicks "Extract".
- **Main Success Scenario:**
    - LLM tool parses raw text using Named Entity Recognition (NER).
    - Agent extracts structured JSON containing item names, quantities, units, and estimated shelf life.
    - Preview table renders in GUI for user review and editing.
    - User clicks "Confirm Import"; items commit to PantryModel.
- **Alternative/Exception Flows:**
    - *A1 (Parsing Error/Garbled Text):* Items flagged in red for manual input correction.
- **Postconditions:** Batch items saved to PantryModel database.
- **Related Features:** F08 — Pantry Import via Receipt / Unstructured Text Parser.

UC09: Export Plan & Sync Calendar

- **Use Case ID:** UC09
- **Use Case Name:** Export Plan & Sync Calendar

- **Actor:** Primary User
- **Goal:** Export active plans and grocery lists to standard external formats (PDF, CSV, .ics).
- **Preconditions:** Meal plan or grocery list generated.
- **Trigger:** User clicks "Export" dropdown menu.
- **Main Success Scenario:**
    - User selects target format (e.g., "Sync to Calendar (.ics)").
    - System transforms meal schedule into standard iCalendar event objects.
    - OS save dialog opens; user saves .ics file locally.
- **Alternative/Exception Flows:**
    - *A1 (File permission denied):* System alerts user and prompts for an alternative target directory.
- **Postconditions:** Exported file written to disk.
- **Related Feature(s):** F09 — Export & Integration Hub.

UC10: Submit Post-Meal Feedback

- **Use Case ID:** UC10
- **Use Case Name:** Submit Post-Meal Feedback
- **Actor(s):** Primary User, External AI Services (LLM)
- **Goal:** Capture rating and critiques on completed recipes to update persistent long-term taste preferences.
- **Preconditions:** Recipe marked as cooked.
- **Trigger:** User submits rating pop-up form.
- **Main Success Scenario:**
    - User submits 1–5 star rating and notes ("Too spicy, reduce chili").
    - Agent extracts preference rule (Dislike: High Spice).
    - UserProfile database updates long-term memory records.
- **Alternative/Exception Flows:**
    - *A1 (Contradictory feedback):* System prompts user to confirm new global preference override.
- **Postconditions:** User profile updated; future agent queries conditioned on new preference rule.
- **Related Feature(s):** F10 — Persistent User Feedback & Taste Profiler.

Features	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Patterns
F01	Dynamic Pantry & Expiry Tracking	Hybrid	UC01	MainGUIView, CLIView, ServiceController, PantryModel	addItem(), updateQuantity(), notifyObservers()	SD03	Observer, MVC

F02	Multi-Day Constrained Plan Generation	AI-Based	UC02	MainGUIView, ServiceController, AgentExecutionEngine, LLMProviderAdapter, PantryLookupTool, PlanStrategy, PlanValidationHandler	handlePlanReq(), executeTask(), execute(), generate(), validate()	SD01	Strategy, Chain of Responsibility, Factory Method, MVC
F03	Iterative Plan Refinement & Single-Meal Swapping	AI-Based	UC03	MainGUIView, ServiceController, AgentExecutionEngine, LLMProviderAdapter, MealPlanModel	handleSwapReq(), executeTask(), reflectAndReplan(), generate()	SD01	Strategy, MVC
F04	Smart Grocery Consolidation & Net Inventory Deduction	Hybrid	UC04	MainGUIView, ServiceController, PantryModel, MealPlanModel, MacroCalcTool	execute(), updateQuantity(), notifyObservers()	SD01, SD03	Observer, Factory Method, MVC
F05	Interactive Real-Time Cooking Mode	AI-Based	UC05	MainGUIView, ServiceController, AgentExecutionEngine, RAGSearchTool, LLMProviderAdapter	handleSubReq(), executeTask(), execute(), generate()	SD02	Factory Method, MVC
F06	Recipe Adaptation & Allergen Sanitizer	AI-Based	UC06	MainGUIView, ServiceController, AgentExecutionEngine, AllergenCheckHandler, RAGSearchTool	executeTask(), validate(), execute(), generate()	SD01, SD02	Chain of Responsibility, Factory Method
F07	Macro & Micronutrient Deep-Dive Analysis	Hybrid	UC07	MainGUIView, ServiceController, MacroCalcTool, LLMProviderAdapter	execute(), generate(), renderPlan()	SD01	Factory Method, MVC
F08	Pantry Import via	AI-Based	UC08	MainGUIView, ServiceController	parseReceipt(), executeTask(),	SD03	Observer, MVC

	Receipt / Text Parser			, AgentExecutionEngine, LLMProviderAdapter, PantryModel	generate(), addItemsToDB(), notifyObservers()		
F09	Export & Integration Hub	Deterministic	UC09	MainGUIView, CLIView, MealPlanModel, ServiceController	generatePlan(), printOutput(), renderPlan()	SD01	MVC
F10	Persistent User Feedback & Taste Profiler	AI-Based	UC10	MainGUIView, ServiceController, AgentExecutionEngine, UserProfile	executeTask(), updatePreference(), generate()	SD01	MVC